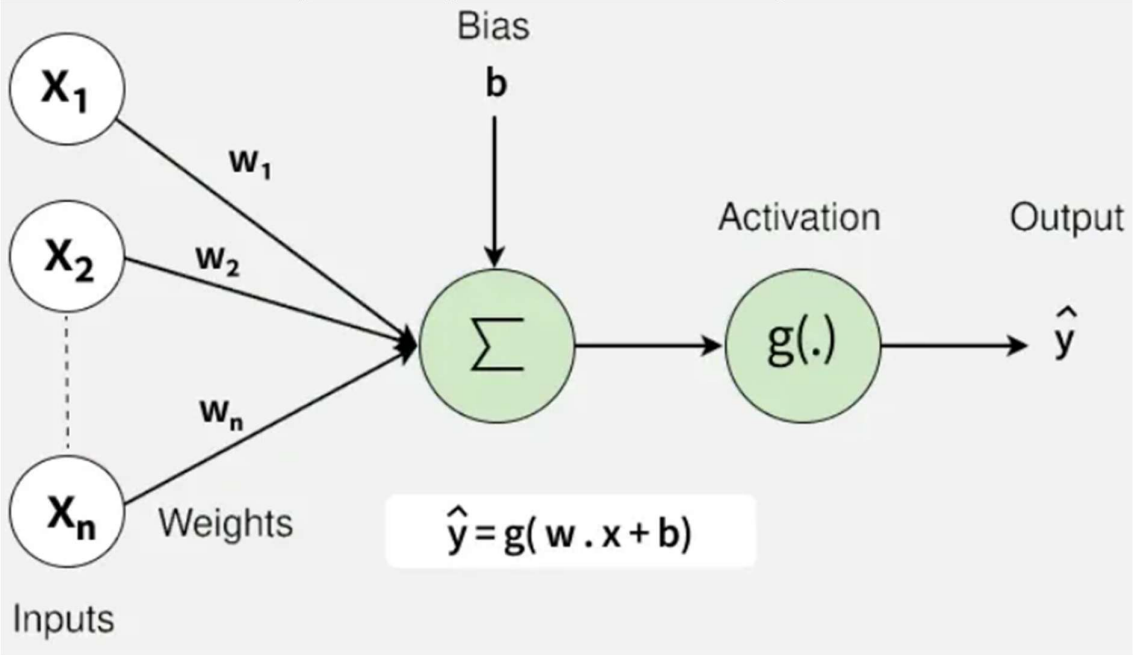
21CSE451T
PATTERN RECOGNITION TECHNIQUES
UNIT-5
MULTILAYER NEURAL NETWORKS AND NONMETRIC METHODS

- **Introduction to Neural Networks**
- **Multilayer Neural Networks**
- **Feedforward Operations and Classification**
- **Backpropagation Algorithms**
- **Nonmetric Methods**
- **Decision Trees**
- **CART**
- **Applications**
    - **Face Recognition System**

INTRODUCTION TO NEURAL NETWORKS

Neural networks are machine learning models that mimic the complex functions of the human brain. These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making.

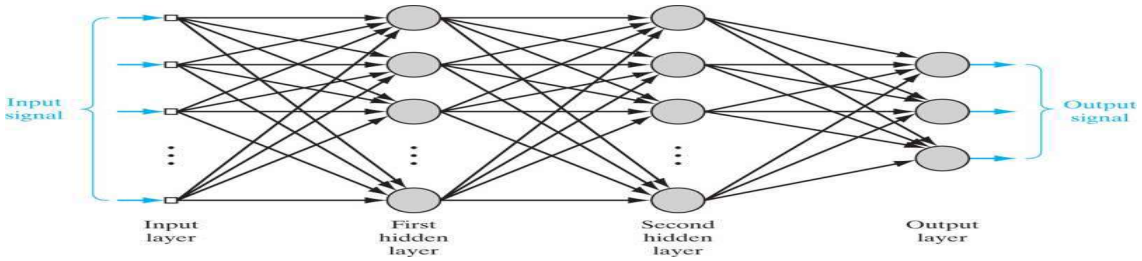


Neurons: The basic units that receive inputs, each neuron is governed by a threshold and an activation function.

Connections: Links between neurons that carry information, regulated by weights and biases.
Weights and Biases: These parameters determine the strength and influence of connections.
Propagation Functions: Mechanisms that help process and transfer data across layers of neurons.
Learning Rule: The method that adjusts weights and biases over time to improve accuracy.

Learning in neural networks follows a structured, three-stage process:
Input Computation: Data is fed into the network.
Output Generation: Based on the current parameters, the network generates an output.
Iterative Refinement: The network refines its output by adjusting weights and biases, gradually improving its performance on diverse tasks.

MULTI-LAYER PERCEPTRON WITH INPUT, HIDDEN AND OUTPUT LAYER



->Each neuron in the network includes a nonlinear activation function that is *differentiable.*
->The network contains one or more layers that are *hidden from* both the input and output nodes.
->The network exhibits a high degree of *connectivity.*

Examples of Using MLP

1.Airline Marketing Tactician
2.Backgammon
3.Data Compression – PCA
4.ECG Noise Filtering
5.Financial Prediction
6.Hand-written Character Recognition
7.Pattern Recognition/Computer Vision
8.Speech Recognition

MULTI-LAYER PERCEPTRON WITH FORWARD

MLP Learning Procedure

- Starting with the input layer, propagate data forward to the output layer. This step is the forward propagation.
- Based on the output, calculate the error (the difference between the predicted and known outcome). The error needs to be minimized.
- Backpropagate the error. Find its derivative with respect to each weight in the network, and update the model.

Repeat the three steps given above over multiple epochs to learn ideal weights.

Finally, the output is taken via a threshold function to obtain the predicted class labels.

Forward Propagation in MLP

In the first step, calculate the activation unit al(h) of the hidden layer.

$$z_1^{(h)} = a_0^{(in)} w_{0,1}^{(h)} + a_1^{(in)} w_{1,1}^{(h)} + \cdots +$$

$$a^{(h)} = \phi(z^{(h)})$$

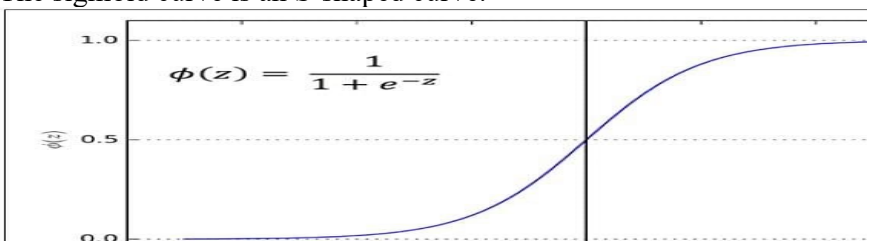
Activation unit is the result of applying an activation function φ to the z value. It must be differentiable to be able to learn weights using gradient descent. The activation function φ is often the sigmoid (logistic) function.

It allows nonlinearity needed to solve complex problems like image processing.

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid Curve

The sigmoid curve is an S-shaped curve.



MULTI-LAYER PERCEPTRON WITH BACKWARD

MULTI-LAYER PERCEPTRON WITH FORWARD AND BACKWARD TO CALCULATE BACK PROPAGATION ERROR

→ Multilayer networks learned by the BACK PROPAGATION algorithm are capable of expressing a rich variety of non-linear decision surfaces.

→ The speech recognition task involves distinguishing among 10 possible vowels, all spoken in the context of "h_d" (i.e. "hid", "had," "head", "hood" etc.).

→ The input speech signal is represented by 2 numerical parameters obtain from a spectral analysis of the sound, allowing us to easily visualize the decision surface over the 2-dimensional instance space

* A) Differentiable Threshold Unit –

→ Multiple layers of cascaded linear units still produce only linear functions but we prefer networks capable of representing highly non-linear functions.

→ So, the perceptron unit is another possible choice, but its discontinuous threshold makes it undifferentiable and hence unsuitable for gradient descent.

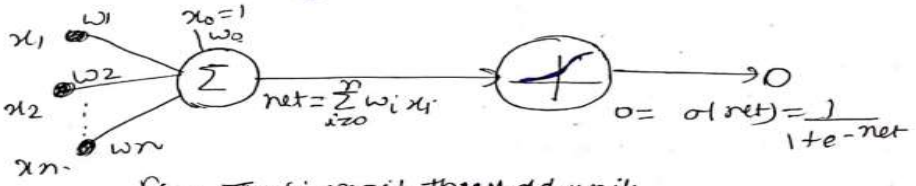


Fig: The Sigmoid threshold unit

→ Output is a non-linear function of its inputs; but whose output is also a differentiable function of its inputs. One solution is the sigmoid unit

- a unit very much like perceptron, but based on smoothed, differentiable threshold function.

(10)

Sigmoid function – Output is a continuous function its input. It computes output o as –

$$o = \sigma(\vec{w} \cdot \vec{x})$$

where, $\sigma(y) = \frac{1}{1+e^{-y}}$

σ is called sigmoid function or logistic function; its output ranges between 0 and 1, increasing monotonically with its input.

→ Because it maps a very input domain to a small range of outputs; it is often referred to as the squashing function of the unit.

* BACKPROPAGATION NETWORKS

→ A backpropagation neural network is a supervised learning feedforward topology that propagates back the errors, adjusting weights till the model is adequately trained.

→ Principle Operation –

- Inputs from the training data are propagated through the network to the output.
- Model outputs are then compared with the actual outputs.
- Difference is the error.
- Error is then fed into the network backwards and corresponding weights adjusted.

... the BACKPROPAGATION

weights adjusted.

→ Stochastic gradient descent version of the BACKPROPAGATION algorithm for feedforward networks containing 2 layers of sigmoid units

BACKPROPAGATION (training_examples, η, n_{in}, n_{out}, n_{hidden})

- Each training example is a pair of the form $\langle \vec{x}, \vec{t} \rangle$, where $\vec{x}$ is the vector of network input values and $\vec{t}$ is the vector of target network output values.
- η is the learning rate (eg. .05).
- n_{in} is the number of network inputs
- n_{hidden} the number of units in the hidden layer
- n_{out} is the number of output units.
- The input from unit i to unit j is denoted by x_{ji}, and weights from unit i to unit j is denoted by w_{ij}.

→ Create a feed-forward network with n_{in} inputs, n_{hidden} hidden units, and n_{out} output units.

→ Initialize all network weights to small random numbers (eg- between −0.05 and 0.05)

→ Until the termination condition is met, do —

→ For each $\langle x, t \rangle$ in training_examples, do (11)

Propagate the input forward through the network

1) Input the instance $\vec{x}$ to the network and compute the output O_u of every unit u in the network.

Propagate the error backward through the network

2) For each network output unit k, calculate its error term δ_k

$$\delta_k \leftarrow O_k(1-O_k)(t_k - O_k)$$

3) For each hidden unit h, calculate its error term δ_h

$$\delta_h \leftarrow O_h(1-O_h) \sum_{k \in outputs} w_{kh}\delta_k$$

4) Update each network weight w_{ji}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

where,

$$\Delta w_{ji} = \eta \delta_j x_{ji} \quad \text{——— (3)}$$

✱ Adding Momentum —

— A variation of BACKPROPAGATION is to alter the weight-update rule in the algo. by making the weight update on the n^{th} iteration depend partially on the update that occurred during the $(n-1)^{th}$ iteration —

$$\Delta w_{ji}(n) = \eta \delta_j x_{ji} + \alpha \Delta w_{ji}(n-1)$$

$\Delta w_{ji}(n)$ is the weight update performed during the n^{th} iteration through the main loop of the algo. and $0 \leq \alpha \leq 1$ is a constant called the momentum.

→ The effect of α is to add momentum that tends to keep the ball rolling in the same direction from one iteration to the next. This can sometimes have the effect of keeping the ball rolling through small local minima in the error surface, or along flat regions in the surface where the ball would stop if there were no momentum.

* Learning in Arbitrary Acyclic Networks –

→ The weight update rule in eqn ③, is retained, and the only change is to the procedure for computing δ values.

→ The δ_r value for a unit r in the layer m is computed from the δ values at the next deeper layer $m+1$ acc. to –

$$\delta_r = O_r(1 - O_r) \sum_{s \in \text{layer } m+1} w_{sr}\delta_s$$

* To generalize the algo, to any directed acyclic graph, the rule for calculating δ for any internal unit (i.e. any unit that is not an output) –

$$\delta_r = O_r(1 - O_r) \sum_{s \in \text{Downstream}(r)} w_{sr}\delta_s$$

Downstream(r) is the set of units immediately downstream from unit r in the network; i.e. all units whose inputs include the output of unit r.

* Derivation of the BACKPROPAGATION Rule –

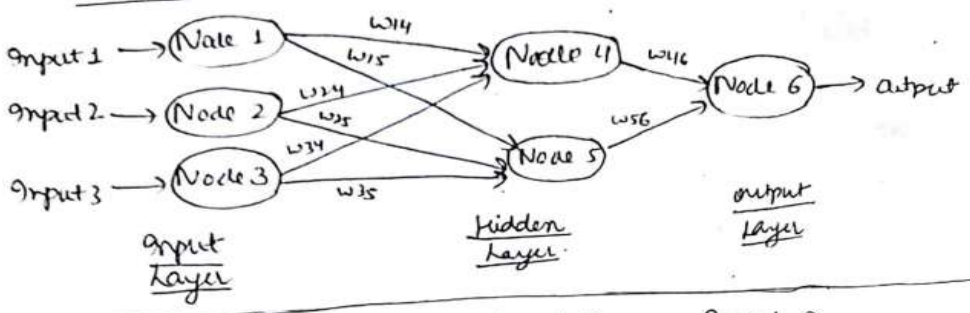
For each training example d every weight w_{ji} is updated by adding to it Δw_{ji}.

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

- Outputs is the set of output units in the network.
- t_k is the target value of unit k for training example d
- O_k is the output of unit k given training example d.

MULTI- LAYER PERCEPTRON IN PRACTICE

BACKPROPAGATION –



Inputs:-	Input 1	Input 2	Input 3
	1.0	0.5	0.1

Arbitrary set of connecting weights:-

w14	w15	w24	w25	w34	w35	w46	w56
1.2	-0.4	0.7	0.6	0.9	1.1	0.8	0.5

1) Input to node 4 = $1(1.2) + 0.5(0.7) + 0.1(0.9) = 1.64$

Output from node 4 = $\frac{1}{1+e^{-1.64}} = 0.8375$

Input to node 5 = $1(-0.4) + 0.5(0.6) + 0.1(1.1) = 0.01$

Output from node 5 = $\frac{1}{1+e^{-0.01}} = 0.5025$

Input to node 6 = $0.8375(0.8) + 0.5025(0.5) = 0.9213$

output from node 6 = $\frac{1}{1+e^{-0.9213}} = 0.7153$

2) Suppose, the actual observed output = 0.6

Error = $0.7153 - 0.6 = 0.1153$

Error = $0.7153 - 0.6 = 0.1153$ in weight w_{46} –

3) Now, propagating backwards error in weight w_{46} –

Error $(w_{46}) = 0.1153 \times [f'(x_6)]$

$f'(x_6) = O_6(1 - O_6)$ f(x) is the derivative of the sigmoid function at node 6

Error $(w_{46}) = 0.1153 \times 0.7153(1 - 0.7153) = 0.023$

Error $(w_{56}) = 0.023$ [same as w_{46}]

(17)

4) Computing the error for internal weights,

$$Error(w_{ij}) = \left[\sum Error(w_{jk})\, w_{jk}\right] f'(x_j)$$

$Error(w_{14}) = 0.023\,(0.8)\,[0.8375\,(1-0.8375)] = 0.00250$

$Error(w_{24}) = 0.0025$ [same as w_{14}]

$Error(w_{34}) = 0.0025$ [same as w_{14}]

$Error(w_{15}) = 0.023\,(0.5)\,[0.5025\,(1-0.5025)]$

$= 0.00287$

$Error(w_{25}) = 0.00287$ (same as w_{15})

$Error(w_{35}) = 0.00287$ [same as w_{15}]

5) Weight Adjustments using Delta Rule developed by Widrow and Hoff (Widrow and Lehr, 1995)

$$w_{jk}(new) = w_{jk}(old) + \Delta w_{jk}$$

$$\Delta w_{jk} = (\eta)\,[Error(w_{jk})]\,[O_k]$$

→ η is the learning rate lies between 0 and 1

→ a higher value means faster convergence but can lead to instability.

Let, $\eta = 0.3$

$\Delta w_{46} = 0.3\,(0.023)\,0.7153 = 0.00494$

$\Delta w_{56} = 0.00494$ [same as Δw_{46}]

updated values,

$w_{46} = 0.8 + 0.00494 = 0.80494$

$w_{56} = 0.5 + 0.00494 = 0.50494$

update values

$\Delta w_{14} = 0.3\,(0.00250)\,0.8375 = 0.000628.$

$\Delta w_{24} = 0.000628$ [Same as Δw_{14}]

$\Delta w_{34} = 0.000628$ [Same as Δw_{14}]

updated value of $w_{14} = 1.2 + 0.000628 = 1.200628.$

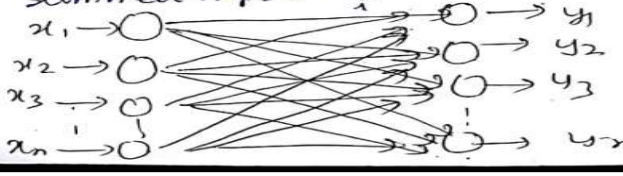
- Similary, update the values of w_{24}, w_{34}, w_{15}, w_{25}, w_{35} can be found.
- We use the updated weights to run the inputs and repeat the whole process in second pass.

* Difference between Single layer Network and Multilayer Network

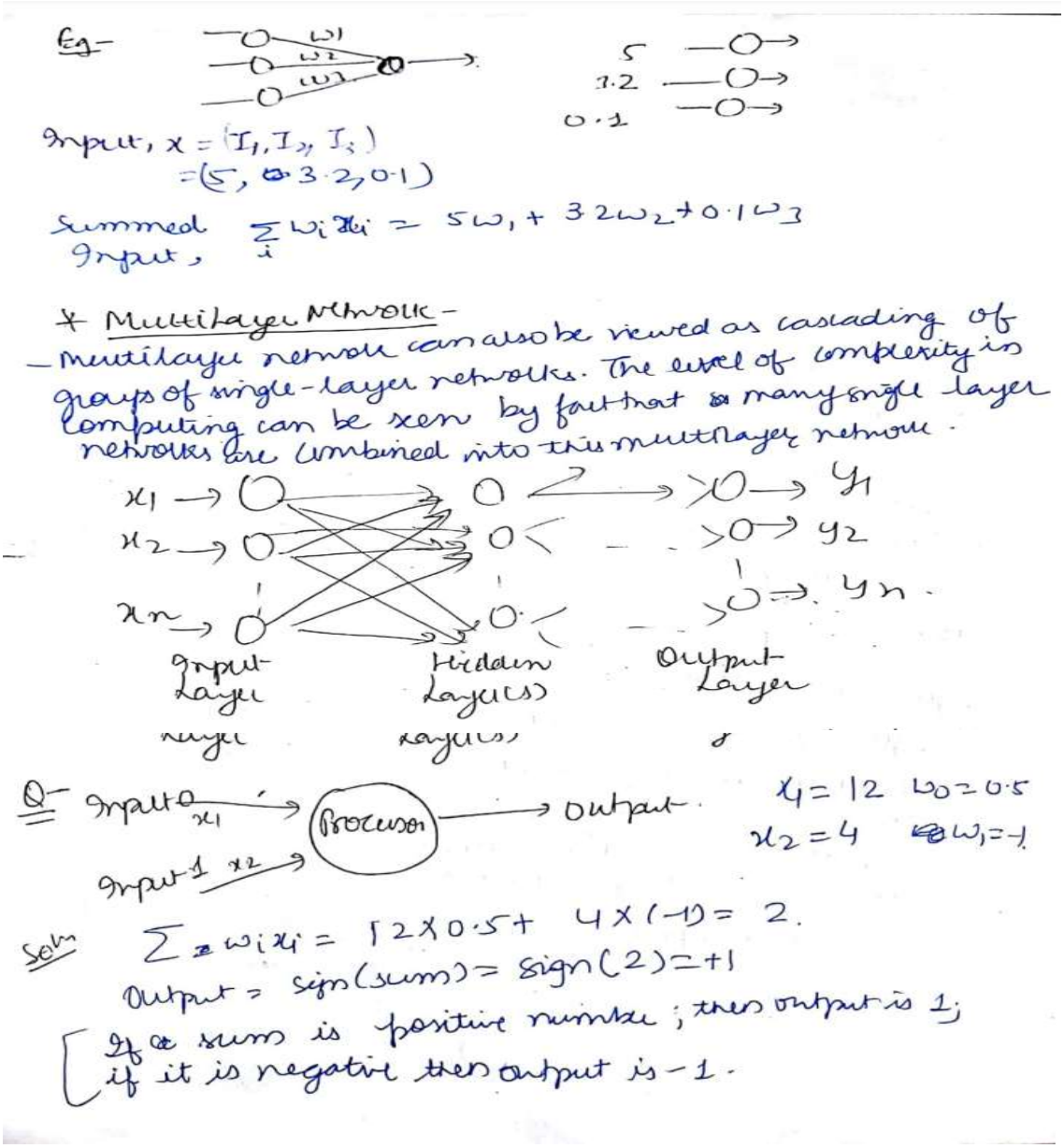
* Single Layer Network –

- Input is a multidimensional (i.e. input can be a vector)
- input $x = [I_1, I_2 \ldots, I_n]$
- Input nodes (or units) are connected to a node in the next lay
- A node in the next layer takes a weighted sum of all its inpu

Summed input $= \sum_i w_i x_i$



The input layer nerv are to only pass and distribute inputs & perform no computation.



EXAMPLES OF USING THE MLP

- Data Compression, Streaming Encoding - Social media, Music Streaming, Online Video Platforms
- Neural Networks for Data Encryption - Data Security / Data Loss Protection
- Autonomous Driving - Image Recognition, Object detection, Route Adjustment
- Customer Ranking - User Profiling - CRM

NON-METRIC METHODS

1. Decision Tree Classifier

Splits feature space using logical conditions.

Example: "If $feature_i$ > threshold → Class A else Class B."
No distance computation.

2. Rule-Based Classifiers
Human-defined or automatically extracted rules.
Example: Expert systems in medical diagnosis (IF–THEN rules).

3. Bayesian Classifiers (Naïve Bayes)
Uses conditional probabilities and Bayes' theorem.
Decision is based on maximum posterior probability, not distances.

4. Artificial Neural Networks (ANNs)
Learn mapping from input features → output classes through layers.
Recognition is based on learned weights and activation functions, not similarity measures.

5. Support Vector Machines (SVMs)
With kernel trick → learns separating hyperplane in high-dimensional space.
Decision is based on margin maximization, not explicit metric comparisons.

6. Random Forests
Ensemble of decision trees.
Voting mechanism decides the class without relying on distance.

DECISION TREES

- Decision trees classify instances by sorting them down the tree from the root to some leaf node, which provides the classification of the instance.
- Each node in the tree specifies a test of some attribute of the instance
- Each branch descending from that node corresponds to one of the possible values for this attribute.
- An instance is classified by starting at the root node of the tree, testing the attribute specified by this node, then moving down the tree branch corresponding to the value of the attribute in the given example. This process is then repeated for the subtree rooted at the new node.

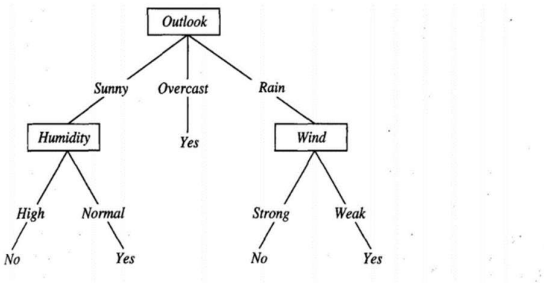


FIGURE: A decision tree for the concept PlayTennis. An example is classified by sorting it through the tree to the appropriate leaf node, then returning the classification associated with this leaf (in this case, Yes or No). This tree classifies Saturday mornings according to whether or not they are suitable for playing tennis.

This decision tree classifies Saturday mornings according to whether they are suitable for playing tennis.
Example:

(Outlook = Sunny, Temperature = Hot, Humidity = High, Wind = Strong)

Each path from the tree root to a leaf corresponds to a conjunction of attribute tests, and the tree itself to a disjunction of these conjunctions.

Example:

(Outlook = Sunny A Humidity = Normal)
v (Outlook = Overcast)
v (Outlook = Rain A Wind = Weak)

APPROPRIATE PROBLEMS FOR DECISION TREE LEARNING

Decision tree learning is generally best suited to problems with the following characteristics:

- **Instances are represented by attribute-value pairs:** Instances are described by a fixed set of attributes (e.g., Temperature) and their values (e.g., Hot). The easiest situation for decision tree learning is when each attribute takes on a small number of disjoint possible values (e.g., Hot, Mild, Cold).
- **The target function has discrete output values:** The decision tree assigns a boolean classification (e.g., yes or no) to each example. Decision tree methods easily extend to learning functions with more than two possible output values.
- **Disjunctive descriptions may be required:** Decision trees naturally represent disjunctive expressions.
- **The training data may contain errors:** Decision tree learning methods are robust to errors, both errors in classifications of the training examples and errors in the attribute values that describe these examples.
- **The training data may contain missing attribute values:** Decision tree methods can be used even when some training examples have unknown values (e.g., if the Humidity of the day is known for only some of the training examples).

CART (Classification and Regression Trees)

- **Classification and regression trees** or **CART** models, also called **decision trees** are defined by recursively partitioning the input space, and defining a local model in each resulting region of input space. This can be represented by a tree, with one leaf per region.
- Classification and Regression Trees or CART for short is a term introduced by Leo Breiman to refer to Decision Tree algorithms that can be used for classification or regression predictive modeling problems.

The model in the following form:

$$f(\mathbf{x}) = \mathbb{E}[y|\mathbf{x}] = \sum_{m=1}^{M} w_m \mathbb{I}(\mathbf{x} \in R_m) = \sum_{m=1}^{M} w_m \phi(\mathbf{x}; \mathbf{v}_m)$$

where, R_m is the m'th region, w_m is the mean response in this region, and $\mathbf{v}_m$ encodes the choice of variable to split on, and the threshold value, on the path from the root to the m'th leaf.

- A CART model is just an adaptive basis-function model, where the basis functions define the regions, and the weights specify the response value in each region.

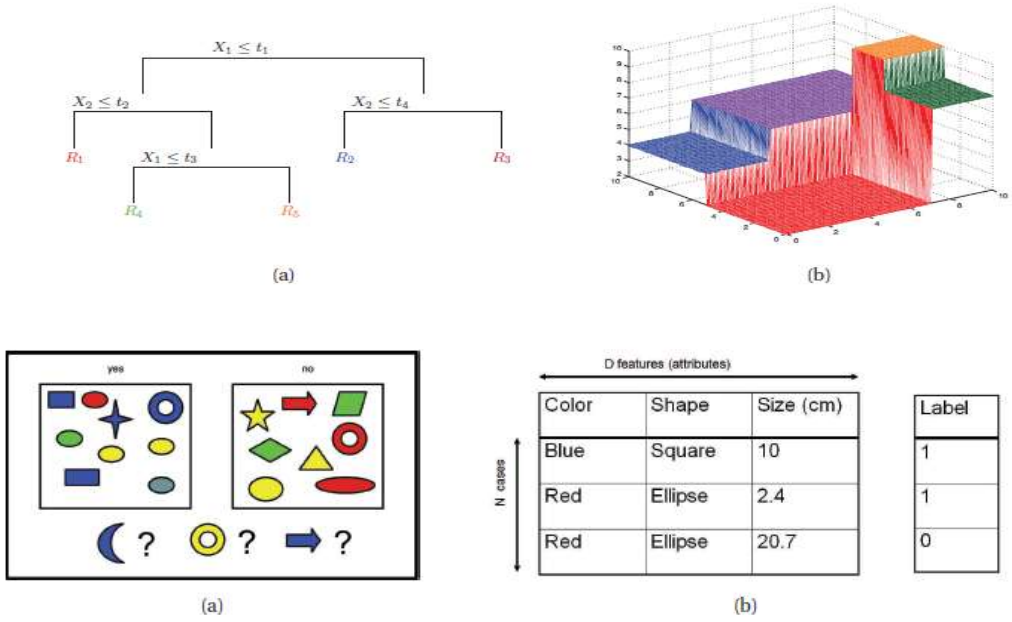


Figure 1.1 Left: Some labeled training examples of colored shapes, along with 3 unlabeled test cases. Right: Representing the training data as an $N \times D$ design matrix. Row i represents the feature vector $\mathbf{x}_i$. The last column is the label, $y_i \in \{0, 1\}$. Based on a figure by Leslie Kaelbling.

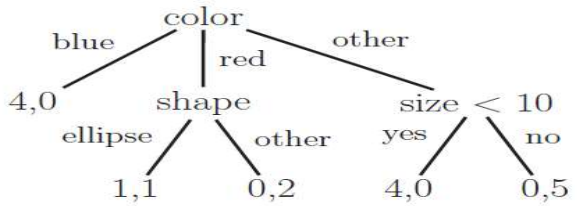


Figure 16.2 A simple decision tree for the data in Figure 1.1. A leaf labeled as (n_1, n_0) means that there are n_1 positive examples that match this path, and n_0 negative examples. In this tree, most of the leaves are "pure", meaning they only have examples of one class or the other; the only exception is leaf representing red ellipses, which has a label distribution of $(1, 1)$. We could distinguish positive from negative red ellipses by adding a further test based on size. However, it is not always desirable to construct trees that perfectly model the training data, due to overfitting.

- If **x** is blue, then $p(y = 1|\mathbf{x}) = 4/4$
- If it is red, we then check the shape: if it is an ellipse, we end up in a leaf labeled "1,1", so we predict $p(y = 1|\mathbf{x}) = 1/2$.
- If it is red but not an ellipse, we predict $p(y = 1|\mathbf{x}) = 0/2$;
- If it is some other colour, we check the size: if less than 10, we predict $p(y = 1|\mathbf{x}) = 4/4$, otherwise $p(y = 1|\mathbf{x}) = 0/5$.

Growing a tree

The split function chooses the best feature, and the best value for that feature, as follows:

$$(j^*, t^*) = \arg \min_{j \in \{1,\ldots,D\}} \min_{t \in \mathcal{T}_j} \text{cost}(\{\mathbf{x}_i, y_i : x_{ij} \leq t\}) + \text{cost}(\{\mathbf{x}_i, y_i : x_{ij} > t\})$$

Algorithm 16.1: Recursive procedure to grow a classification/ regression tree

```
function fitTree(node, D, depth) ;
node.prediction = mean(y_i : i ∈ D) // or class label distribution ;
(j*, t*, D_L, D_R) = split(D);
if not worthSplitting(depth, cost, D_L, D_R) then
    return node
else
    node.test = λx.x_j* < t* // anonymous function;
    node.left = fitTree(node, D_L, depth+1);
    node.right = fitTree(node, D_R, depth+1);
    return node;
```

- If inputs are *real-valued or ordinal*, compare a feature x_{ij} to a numeric value t. The set of possible thresholds Tj for feature j can be obtained by sorting the unique values of x_{ij}. For example, if feature 1 has the values {4.5,−12, 72,−12}, then we set T_1 = {−12, 4.5, 72}.
- In the case of *categorical inputs*, the most common approach is to consider splits of the form $x_{ij} = c_k$ and $x_{ij} != c_k$, for each possible class label c_k.

The function that checks if a node is worth splitting can use several stopping heuristics, such as the following:

- is the reduction in cost too small?
- has the tree exceeded the maximum desired depth?
- is the distribution of the response in either D_L or D_R sufficiently homogeneous (e.g., all labels are the same, so the distribution is **pure**)?
- is the number of examples in either D_L or D_R too small?

1) Regression cost

$$\text{cost}(\mathcal{D}) = \sum_{i \in \mathcal{D}} (y_i - \overline{y})^2$$

$$\text{where } \overline{y} = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} y_i$$

is the mean of the response variable in the specified set of data.

2) Classification Cost

Fit a model to the data in the leaf satisfying the test Xj < t by estimating the class-conditional probabilities as follows:

$$\hat{\pi}_c = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \mathbb{I}(y_i = c)$$

where $\mathcal{D}$ is the data in the leaf.

Error measures for evaluating a proposed partition

- **Misclassification rate**: The most probable class label as $\hat{y}_c = \text{argmax}_c \, \hat{\pi}_c$. The corresponding error rate is then:

$$\frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}}\mathbb{I}(y_i \neq \hat{y}) = 1 - \hat{\pi}_{\hat{y}}$$

- **Entropy**, or **deviance**:

$$\mathbb{H}(\hat{\boldsymbol{\pi}}) = -\sum_{c=1}^{C}\hat{\pi}_c \log \hat{\pi}_c$$

$\hat{\pi}_c$ is an MLE for the distribution $p(c|X_j < t)$

- **Gini index**

$$\sum_{c=1}^{C}\hat{\pi}_c(1-\hat{\pi}_c) = \sum_c \hat{\pi}_c - \sum_c \hat{\pi}_c^2 = 1 - \sum_c \hat{\pi}_c^2$$

$\hat{\pi}_c$ is the probability a random entry in the leaf belongs to class c, and $(1-\hat{\pi}_c)$ is the probability it would be misclassified

Pros of Trees

- Easy to interpret
- Easily handle mixed discrete and continuous inputs
- Insensitive to monotone transformations of the inputs (because the split points are based on ranking the data points)
- Perform automatic variable selection
- Relatively robust to outliers
- Scale well to large data sets
- Modified to handle missing inputs

Cons of Trees

- Do not predict very accurately compared to other kinds of model.
- Trees are **unstable**
- Small changes to the input data can have large effects on the structure of the tree
- Due to the hierarchical nature of the tree-growing process, causing errors at the top to affect the rest of the tree.
- Trees are high variance estimators.

APPLICATIONS: FACE RECOGNITION SYSTEM

A face recognition system is a specialized application of pattern recognition, where the "patterns" are human faces.

1. Pattern Recognition Basics: Pattern recognition is the science of detecting and classifying data (signals, images, speech, etc.) based on statistical information and structural features.
Process: It involves data acquisition → feature extraction → classification.

2. Face Recognition as Pattern Recognition

Input Pattern: An image (2D or 3D) of a human face.
Feature Extraction: Identifying unique patterns such as:

Geometric features (distance between eyes, nose shape, jawline).
Texture patterns (skin tone, wrinkles, edges).
Encoded deep features (using CNN, GAT, or embeddings).

Pattern Classification:

The extracted features are matched with stored patterns in a database.
Classifiers (like SVM, k-NN, or deep learning models) recognize whether the face belongs to a known individual.

3. Steps in Face Recognition System

Face Detection → Locate the face region in an image (e.g., Viola-Jones, MTCNN, YOLO).
Feature Extraction → Represent the face with unique patterns (Eigenfaces, Fisherfaces, deep embeddings like FaceNet).
Pattern Classification → Compare features with a database using similarity metrics or classifiers.
Decision → Accept, reject, or identify the person.

4. Techniques in Pattern Recognition for Face Recognition

Statistical Methods: PCA (Eigenfaces), LDA (Fisherfaces).
Machine Learning: SVM, Random Forest, k-NN.
Deep Learning: CNNs (VGG-Face, ResNet, FaceNet), Transformers, GAT-based embeddings.
Hybrid Models: Combine temporal or structural data (e.g., FAVAR-GAT for dynamic features).

5. Applications

Security & surveillance (access control, border security).
Mobile authentication (Face ID).
Forensics & law enforcement.
Personalized services (e.g., tagging in social media, customer analytics).